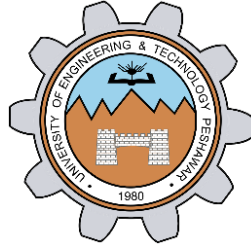


RUSHD AI: Revolutionizing Understanding in Students and Helping Digital Academic Intelligence



2024-25

Submitted By

Ahsan Raza

22PWCSE2099

Muhammad Saad Amjad Khan

22PWCSE2133

Hassan Zaib Jadoon

22PWCSE2144

Supervised by

Dr. Madiha Sher

Department of Computer Systems Engineering
University of Engineering and Technology, Peshawar



UNIVERSITY OF ENGINEERING AND TECHNOLOGY, PESHAWAR, PAKISTAN

B.Sc PROJECT PROPOSAL

Proposal Title	RUSHD AI: Revolutionizing Understanding in Students and Helping Digital Academic Intelligence		
Department	Computer Systems Engineering		
Registration No.	Student Name	Email	Contact No.
22pwcse2144	Hassan Zaib Jadoon	22pwcse2144@uetpeshawar.edu.pk	03119541429
22PWCSE2099	Ahsan Raza	22pwcse2099@uetpeshawar.edu.pk	03197529839
22PWCSE2133	Muhammad Saad Amjad Khan	22pwcse2133@uetpeshawar.edu.pk	03079966105
Research Supervisor	Dr. Madiha Sher		

Mapping the Project with Computer Systems Engineering Subjects

(Subjects that you have studied till now – Please map at least 5 courses from your transcript)

S. No.	Course Code	Course Name	Possible linkage with the project (Justify)
1.	(CSE-210)	Data Structures and Algorithms	• The project relies heavily on efficient data handling, parsing, and algorithmic logic for tasks like summarization, flashcard generation, and quiz creation. • Core concepts of searching, sorting, tree structures, and graph-based models are applicable in implementing NLP pipelines and backend modules.
2.	(CSE-403)	Database Management System	• Storage and retrieval of processed content (summaries, rubrics, flashcards, quizzes) require structured and optimized database design. • Concepts such as indexing, query optimization, and schema design directly support the MongoDB/Firebase backend of RUSHD AI.
3.	(CSE-204)	Operating Systems	• Understanding process management, memory handling, and concurrent operations helps in designing efficient backend services (Flask/Django). • The system's ability to handle multiple simultaneous requests from teachers and students benefits from OS-level knowledge.
4.	(CSE-308)	Digital System Design	• Concepts of sequential and combinational design thinking are applied in structuring workflows like input extraction → NLP processing → content output.
5.	(CSE-208)	Object Oriented Programming	• The project's modular architecture (backend services, NLP modules, user interface) is implemented using OOP principles such as encapsulation, inheritance, and abstraction. • Python classes and reusable components form the foundation of the system.

Introduction

In the modern academic landscape, both students and teachers are required to interact with a vast and continuously growing pool of educational resources such as textbooks, research articles, lecture slides, online videos, and web-based content. While the availability of such resources enriches the learning environment, it also creates challenges in terms of information overload, knowledge retention, and time management. Students often struggle to filter out key concepts, retain critical knowledge, and prepare efficiently for assessments, whereas teachers face significant challenges in designing lectures, preparing exam papers, and generating fair rubrics within limited time frames. These challenges not only reduce learning efficiency but also impact the overall quality of teaching and assessment.

Artificial Intelligence (AI) and Natural Language Processing (NLP) have emerged as transformative tools in addressing these academic challenges. Recent advancements in machine learning, automated summarization, and content generation have demonstrated potential in reducing manual effort, enhancing comprehension, and supporting interactive learning. However, existing tools are often fragmented — most solutions are either focused solely on students (summarization tools, flashcards, quizzes) or designed primarily for teachers (exam generation, rubric tools). There remains a gap for an integrated, intelligent system that serves both learners and educators simultaneously in an interactive and adaptive manner.

RUSHD AI — Revolutionizing Understanding in Students and Helping Digital Academic Intelligence — is proposed as a unified AI-powered learning assistant to bridge this gap. The system is designed to process diverse content formats such as PDF, DOCX, PPTX, research articles, YouTube videos, and web pages, applying NLP and ML techniques to generate summaries, flashcards, key questions, equations, and interactive quizzes for students. For teachers, it offers features like rubric generation, lecture preparation, and exam creation with adjustable difficulty levels, thereby reducing repetitive workload and enhancing teaching effectiveness. By combining both student assistance and teacher support within a single platform, RUSHD AI aims to foster an efficient, interactive, and intelligent learning ecosystem that aligns with the needs of modern education.

Problem Statement:

Modern education involves vast resources for both students and teachers. Students often struggle to extract and retain key information, while teachers spend significant time creating rubrics, exams, and lectures. RUSHD AI aims to address these challenges by providing intelligent assistance to both students and teachers, offering interactive summaries, flashcards, quizzes, as well as rubric generation and lecture preparation.

Methodology:

The development of RUSHD AI will follow a structured and iterative methodology to ensure both academic rigor and practical usability. The process begins with requirement analysis and literature

review, where the needs of both students and teachers will be carefully studied and compared against existing solutions in AI-driven educational technology. This phase will help define the system's functional and non-functional requirements, ensuring that it addresses the gaps in current tools. Following this, the system design and architecture will be developed, including the creation of data flow diagrams, module specifications, and user interface wireframes for both student and teacher pipelines. Once the design is finalized, data collection and preprocessing will be carried out, where documents, research papers, presentations, and YouTube videos will be processed using libraries such as PyMuPDF, python-docx, python-pptx, and Google Speech-to-Text API. The extracted data will undergo preprocessing through tokenization, stop-word removal, lemmatization, and sentence segmentation using tools like NLTK and spaCy to prepare it for NLP tasks.

The next stage will focus on integrating NLP and machine learning models to generate summaries, key points, flashcards, questions, and rubrics. Pretrained models such as BERT and T5 from HuggingFace Transformers will be employed and fine-tuned for educational content to improve accuracy, while custom algorithms will be implemented for equation detection and principle extraction. In parallel, backend development using Flask or Django will provide robust APIs for content processing, and the frontend, built with ReactJS, will deliver an interactive user interface for both students and teachers. A database system, preferably MongoDB or Firebase, will be used to store processed content such as quizzes, flashcards, and rubrics, ensuring optimized data retrieval and personalized dashboards for users.

Once the system is functional, testing and evaluation will be carried out at multiple levels, including unit testing for individual modules, integration testing for pipeline coordination, and user acceptance testing with sample groups of students and teachers. The performance of NLP outputs will be validated using metrics such as ROUGE, BLEU, and F1 scores. After thorough evaluation, the system will be deployed on a cloud-based server like AWS, GCP, or Heroku for scalability and accessibility. Finally, complete documentation, including a user manual and technical report, will be prepared alongside the demonstration of a fully functional Minimum Viable Product (MVP).

Equipment/ Tools Usage:

- Language: Python
- Backend: Flask / Django
- Frontend: HTML, CSS, JavaScript, ReactJS
- Libraries: HuggingFace Transformers, spaCy, NLTK, PyMuPDF, python-docx, python-pptx
- APIs: YouTube Data API, Google Speech-to-Text
- Database: MongoDB / Firebase

References:

[1] J. Devlin et al., "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," NAACL, 2019.

[2] C. Raffel et al., "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," arXiv:1910.10683, 2019.

[3] T. Karpicke and J. Blunt, "Retrieval Practice Produces More Learning than Elaborative Studying with Concept Mapping," Science, vol. 331, no. 6018, 2011.

[4] HuggingFace, "Transformers," Available: <https://huggingface.co/transformers/>.

[5] Google Developers, "YouTube Data API v3," Available: <https://developers.google.com/youtube/v3>